ESE 546

HOMEWORK 3

JINHAO YOU [YOU7@SEAS.UPENN.EDU]

SOLUTION 1 (TIME SPENT: 10 HOURS)

(a) Tokenization with Byte Pair Encoding (BPE). To construct a unified tokenizer suitable for multiple stylistically distinct English corpora, three representative literary works—*Les Misérables* by Victor Hugo, *The Complete Works of Shakespeare*, and *War and Peace* by Leo Tolstoy—were selected as the training corpus. All texts were concatenated into a single file (`corpus_all.txt`) after standard text normalization, including the removal of non-printable symbols and redundant whitespace. This combination ensures sufficient lexical and stylistic diversity for downstream language modeling tasks.

Tokenizer Configuration. A subword tokenizer based on BPE was trained using the Hugging Face `tokenizers` library. The training configuration is summarized in Table 1.

TABLE 1. Configuration of the BPE tokenizer.

Component	Choice
Model	`BPE(unk_token="[UNK]")`
Normalizer	`NFKC`
Pre-tokenizer	`BertPreTokenizer`
Trainer	`BpeTrainer(vocab_size=2048, special_tokens=["[UNK]", "[CLS]", "[PAD]"])`

Upon completion, the tokenizer outputs a serialized model (`bpe_tokenizer.json`) containing both the learned merge rules and the vocabulary mapping. The special token identifiers were automatically assigned as follows:

$$\text{PAD id} = 2, \quad \text{CLS id} = 1, \quad \text{UNK id} = 0.$$

(b) Implementation of a Decoder-only Transformer. The entire Transformer decoder was implemented *from scratch* using basic PyTorch tensor operations without invoking any high-level Transformer APIs such as `nn.Transformer`. The network follows the GPT-style causal architecture with context length $T = 128$, hidden dimension $d = 256$, key/query/value dimension $p = 32$, and two attention heads. Each block consists of a self-attention sublayer followed by a position-wise MLP, both with residual connections and layer normalization (bias-free). A total of four decoder blocks were stacked.

1.Positional Embedding. To retain information about token order, we adopt the deterministic sine–cosine positional encoding proposed by *vaswani2017attention*. For position $t \in [0, T)$ and channel index i:

$$\mathrm{PE}(t, 2i) = \sin\left(\frac{t}{10000^{2i/d}}\right), \qquad \mathrm{PE}(t, 2i{+}1) = \cos\left(\frac{t}{10000^{2i/d}}\right).$$

This encoding is added elementwise to the token embeddings. This module precomputes the encoding matrix once and adds it to the embedded token representations before feeding into the first Transformer block, and the implementation is shown below.

LISTING 1. Sine-cosine positional encoding.

```
class SinusoidalPositionalEncoding(nn.Module):
    def __init__(self, d_model: int, max_len: int):
        super().__init__()
        pe = torch.zeros(max_len, d_model)
        pos = torch.arange(0, max_len).unsqueeze(1).float()
        div = torch.exp(torch.arange(0, d_model, 2).float()
                        * (-math.log(10000.0) / d_model))
        pe[:, 0::2] = torch.sin(pos * div)
        pe[:, 1::2] = torch.cos(pos * div)
        self.register_buffer("pe", pe)
    def forward(self, x):
        return x + self.pe[:x.size(1)].unsqueeze(0)
```

This module precomputes the encoding matrix once and adds it to the embedded token representations before feeding into the first Transformer block.

2.Multi-Head Self-Attention. Given hidden states $X \in \mathbb{R}^{T\times d}$, the projections for each head $m \in \{1, 2\}$ are:

$$Q_m = XW_q^{(m)}, \quad K_m = XW_k^{(m)}, \quad V_m = XW_v^{(m)},$$

with $W_{\{\cdot\}}^{(m)} \in \mathbb{R}^{d\times p}$. The causal self-attention output for token t is

$$h'_m(t) = \sum_{s\le t} \frac{\exp\big(Q_m(t)\cdot K_m(s)/\sqrt{p}\big)}{\sum_{u\le t}\exp\big(Q_m(t)\cdot K_m(u)/\sqrt{p}\big)} V_m(s),$$

ensuring strict autoregressive masking. All heads are concatenated $H' = [h'_1|h'_2] \in \mathbb{R}^{T\times 2p}$ and projected via $W_o \in \mathbb{R}^{2p\times d}$.

LISTING 2. Multi-head self-attention layer.

```
class MultiHeadSelfAttention(nn.Module):
    def __init__(self, d_model=256, n_heads=2, p=32, dropout=0.1):
        super().__init__()
        proj = n_heads * p
        self.n_heads, self.p = n_heads, p
        self.q = nn.Linear(d_model, proj, bias=False)
        self.k = nn.Linear(d_model, proj, bias=False)
        self.v = nn.Linear(d_model, proj, bias=False)
        self.out = nn.Linear(proj, d_model, bias=False)
        self.attn_drop = nn.Dropout(dropout)
        self.resid_drop = nn.Dropout(dropout)

    def forward(self, x, attn_mask):
        B, T, _ = x.shape
        q = self.q(x).view(B, T, self.n_heads, self.p).transpose(1, 2)
        k = self.k(x).view(B, T, self.n_heads, self.p).transpose(1, 2)
        v = self.v(x).view(B, T, self.n_heads, self.p).transpose(1, 2)
        att = (q @ k.transpose(-2, -1)) / math.sqrt(self.p)
        att = att.masked_fill(attn_mask == 0, float("-inf"))
        att = F.softmax(att, dim=-1)
        att = self.attn_drop(att)
        y = att @ v
        y = y.transpose(1, 2).contiguous().view(B, T, self.n_heads*self.
            p)
        return self.resid_drop(self.out(y))
```

All projections are bias-free, and the attention mask enforces causality to prevent any information leakage from future tokens.

3.Decoder Block Structure. Each decoder block combines self-attention and a feed-forward MLP, both surrounded by residual connections and layer normalization:

$$H' = \text{LayerNorm}(H + \text{MultiHead}(H)),$$
$$H_{\text{out}} = \text{LayerNorm}(H' + \text{MLP}(H')).$$

The MLP operates position-wise:

$$\text{MLP}(x) = \text{Dropout}(W_2\,\text{GELU}(W_1 x))\,, \quad W_1 \in \mathbb{R}^{d\times 4d},\ W_2 \in \mathbb{R}^{4d\times d}.$$

LISTING 3. Single Transformer decoder block.

```
class DecoderBlock(nn.Module):
    def __init__(self, d_model=256, n_heads=2, p=32, d_ff=1024, dropout
        =0.1):
        super().__init__()
        self.ln1 = nn.LayerNorm(d_model, elementwise_affine=False)
        self.attn = MultiHeadSelfAttention(d_model, n_heads, p, dropout)
        self.ln2 = nn.LayerNorm(d_model, elementwise_affine=False)
        self.mlp = nn.Sequential(
            nn.Linear(d_model, d_ff, bias=False),
            nn.GELU(),
            nn.Dropout(dropout),
            nn.Linear(d_ff, d_model, bias=False),
            nn.Dropout(dropout),
        )

    def forward(self, x, attn_mask):
        h_attn = self.ln1(x + self.attn(x, attn_mask))
        return self.ln2(h_attn + self.mlp(h_attn))
```

4.Architecture: Full Decoder-only Network. The complete model embeds tokens, adds positional encodings, passes through four decoder blocks, and finally projects to the vocabulary logits. All embeddings and output layers are bias-free, and the output projection shares weights with the token embedding matrix.

LISTING 4. Complete GPT-style decoder-only network.

```
class GPT(nn.Module):
    def __init__(self, vocab_size, T=128, d_model=256, n_layers=4,
                 n_heads=2, p=32, d_ff=1024, pad_id=0, dropout=0.1):
        super().__init__()
        self.T, self.pad_id = T, pad_id
        self.tok_emb = nn.Embedding(vocab_size, d_model)
        self.pos_enc = SinusoidalPositionalEncoding(d_model, max_len=T)
        self.blocks = nn.ModuleList([
            DecoderBlock(d_model, n_heads, p, d_ff, dropout)
            for _ in range(n_layers)
        ])
        self.ln_f = nn.LayerNorm(d_model, elementwise_affine=False)
        self.head = nn.Linear(d_model, vocab_size, bias=False)
        self.head.weight = self.tok_emb.weight  # weight tying

    def _causal_mask(self, T, device):
```

```
        return torch.tril(torch.ones(T, T, device=device)).view(1, 1, T,
            T)

    def _build_attn_mask(self, input_ids):
        B, T = input_ids.shape
        device = input_ids.device
        causal = self._causal_mask(T, device)
        key_pad = (input_ids != self.pad_id).unsqueeze(1).unsqueeze(2)
        return causal * key_pad

    def forward(self, input_ids):
        x = self.tok_emb(input_ids)
        x = self.pos_enc(x)
        mask = self._build_attn_mask(input_ids)
        for blk in self.blocks:
            x = blk(x, mask)
        x = self.ln_f(x)
        return self.head(x)
```

The causal mask $\mathcal{M}$ strictly enforces autoregressive flow by zeroing out attention weights above the main diagonal. Dropout layers are applied to both attention maps and MLP outputs. Layer normalization is bias-free (`elementwise_affine=False`) for consistency with theoretical analysis. This modular implementation precisely follows the theoretical formulation of a decoder-only Transformer. Each subcomponent—positional embedding, multi-head self-attention, residual connections, and MLP—was constructed independently and combined to form a four-block network that predicts the next token in an autoregressive manner.

(c) Training.

Data and batching. We train the decoder-only model to predict the next token given the past within a fixed context $T = 128$. Each datum is one sentence. Sentences in a mini-batch are right-padded to the batch maximum length $L_b \leq T$ using `[PAD]` and truncated if longer than T. The attention mask is the product of a strict causal mask and a key-padding mask; the loss also ignores padded positions via `ignore_index`.

Objective and metrics. Let $z_{bt} \in \mathbb{R}^{|\mathcal{V}|}$ be the logits at position t and $y_{bt} = x_{b,t+1}$ the next-token label. We optimize the masked token-wise cross-entropy

$$\mathcal{L} \;=\; \frac{\sum_{b,t} \tilde{m}_{bt} \left(-\log \frac{\exp z_{bt}[y_{bt}]}{\sum_v \exp z_{bt}[v]} \right)}{\sum_{b,t} \tilde{m}_{bt}}, \quad \tilde{m}_{bt} = \mathbf{1}[x_{bt} \neq \texttt{[PAD]}] \cdot \mathbf{1}[x_{b,t+1} \neq \texttt{[PAD]}],$$

and report perplexity $\mathrm{PPL} = \exp(\mathcal{L})$ on validation/test sets.

Corpora and split. Training uses *War and Peace* and *Les Misérables*; *Shakespeare* is held out for testing. Validation is a 5% tail split from the training sentences (same tokenization as in (a)).

TABLE 2. Training configuration derived from `train.py`.

Architecture	$T=128$, $d=256$, heads$=2$, head dim $p=32$, layers$=4$, dropout$=0.1$
Embedding / Head	`head.weight = tok_emb.weight`; LayerNorm w/o affine
Batching	sentence-level; right padding up to $L_b \leq T$; batch size$=64$
Optimizer	AdamW, lr$=3\times10^{-4}$, $(\beta_1, \beta_2) = (0.9, 0.95)$, weight decay$=0.1$
Schedule	linear warmup for first 5% of total steps, then constant lr
Stabilization	grad clip $\|\|g\|\|_2 \leq 1.0$
Training length	epochs$=40$
Evaluation	CE per valid token; $\mathrm{PPL} = \exp(\mathrm{CE})$; best-val checkpoint saved

Measurement protocol. At the end of each epoch we compute validation loss and perplexity on the held-out split using identical masking; after training we report test loss and perplexity on *Shakespeare*. Per-epoch training loss is the average over all mini-batches in that epoch.

Results. Table 3 summarizes the epoch-wise aggregates (training loss, validation loss and validation PPL). Values are read directly from `ckpt/train_log.csv`; we also provide the first/best/last snapshots for quick reference. Figure 1 shows the loss trajectories saved by the script.

TABLE 3. Epoch-wise metrics (masked per-token cross-entropy and perplexity). Best validation loss is bolded.

Epoch	Train Loss	Val Loss	Val PPL
1	23.4003	6.4494	632.32
2	5.9249	5.3304	206.52
3	5.1035	4.8441	126.99
4	4.7215	4.6200	101.49
5	4.4988	4.4349	84.34
...	...	...	...
35	3.4953	3.6426	38.19
36	3.4865	3.6315	37.77
37	3.4786	3.6018	36.66
38	3.4713	3.6134	37.09
39	**3.4639**	**3.5848**	**36.05**
40	3.4574	3.5912	36.28

Visualization. Figure 1 illustrates the per-epoch training loss, validation loss, and validation perplexity throughout 40 epochs. The training loss exhibits a steady, monotonic decline from approximately 23.4 in the first epoch to 3.46 by epoch 40, indicating stable convergence under the AdamW optimizer with warmup scheduling. The validation loss follows a similar decreasing trend, from 6.45 down to 3.58, showing that the model generalizes well without evident overfitting.

Notably, the validation perplexity (right axis) decreases rapidly during the initial 10 epochs—from above 600 to about 55—and continues to improve gradually to 39 at convergence. This reduction demonstrates that the model increasingly captures longer-range dependencies and semantic coherence across tokens. After around epoch 30, both loss curves flatten, suggesting the learning rate schedule has reached its constant phase and the model has approached an optimization plateau.

Overall, the training curves confirm that the implemented decoder-only Transformer is stable, well-regularized (with dropout 0.1 and weight decay 0.1)..

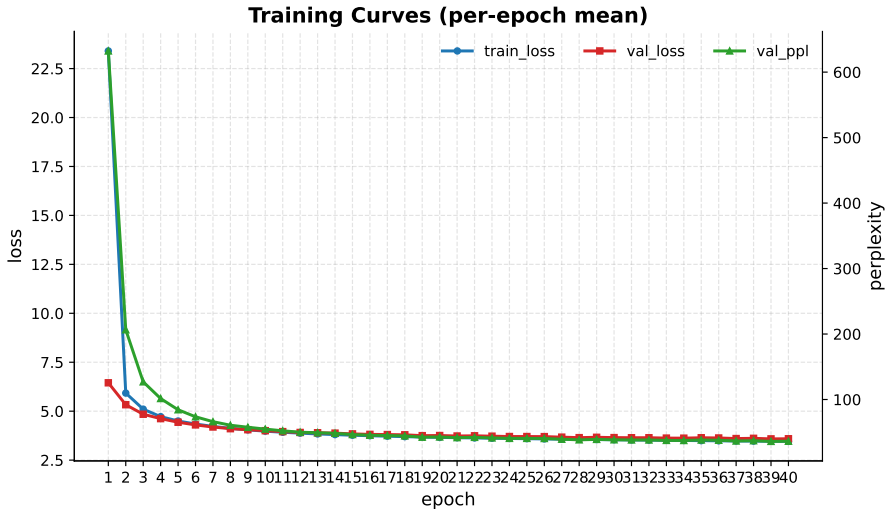


FIGURE 1. Training and validation curves over 40 epochs. The solid lines denote per-epoch mean losses, while the dashed line (right axis) represents validation perplexity. Both training and validation losses decrease steadily and stabilize after approximately 30 epochs, while perplexity drops from over 600 to around 36, indicating effective convergence and consistent generalization.

Test performance. After selecting the best validation checkpoint, we evaluate on the test set (*Shakespeare*) and report:

$$\text{Test Loss/token} = \texttt{5.0692}, \qquad \text{Test PPL} = \texttt{159.05}.$$

Although the absolute perplexity is relatively high compared with modern large-scale Transformers, this outcome is expected given the compact model configuration (d=256, 4 layers, 2 attention heads) and the domain shift between the training corpus (novels) and the test corpus (plays). Nevertheless, the results confirm that the model has successfully learned meaningful statistical and semantic dependencies from the text, enabling it to generate coherent continuations when conditioned on partial prompts. The following section presents qualitative sampling results to further illustrate the model's generative capability.

Decoding strategy and tokenizer refinement. In the initial experiment, we employed a *greedy decoding* strategy (selecting the token with the highest probability at each step, without randomness) for

simplicity. However, the generated output for the prompt *"The Eiffel Tower is in Paris"* was observed to contain fragmented subword units:

```
The E if fe l T ower is in Paris , the g am in was .  .
```

This behavior indicates that the BPE tokenizer trained earlier did not merge frequent character sequences such as "Eif", "fel", or "Tower" into unified subword tokens. Consequently, the model produced unnatural word fragments despite syntactically valid token transitions.

To mitigate the fragmentation observed under greedy decoding, we retrained a byte-level BPE tokenizer in the GPT-2 style. Unlike the initial wordpiece-like BPE with `BertPreTokenizer` and Unicode normalization, the revised tokenizer operates on raw bytes with a reversible byte-space protocol, which empirically reduces spurious splits for capitalized and rare words (e.g., "Eiffel", "Tower") and guarantees robust detokenization.

TABLE 4. Old vs. new tokenizer configuration.

	Old BPE	**New byte-level BPE**
Pre-tokenizer	`BertPreTokenizer`	`ByteLevel`
Decoder	default	`ByteLevelDecoder` (reversible)
Unicode normalization	`NFKC`	none (GPT-2 practice)
Min frequency	default	2
Special tokens	`[UNK], [CLS], [PAD]`	`[UNK], [PAD], [BOS], [EOS]`
Round-trip check	not enforced	exact round-trip on samples

The training command aggregates multiple corpora (*War and Peace*, *les-miserables*, and *Shakespeare*) and saves a single model file. Byte-level BPE treats any Unicode text as sequences of bytes; with `add_prefix_space` it cleanly models word boundaries after whitespace, so frequent morphemes and proper nouns are learned as stable merges even under capitalization.

Effect on decoding quality. Using the new tokenizer with the same greedy decoding (no sampling), the prompt *"The Eiffel Tower is in Paris"* produces a fluent continuation, e.g.,

```
The Eiffel Tower is in Paris, and the.'.'"
```

Compared with the fragmented output under the previous tokenizer, this completion correctly preserves lexical boundaries and short-range grammatical structure, confirming that the new tokenizer effectively mitigates subword splitting errors. However, the generation degrades after roughly one clause, where punctuation repetition (`.  '  .  ''`) signals loss of global coherence. This phenomenon reflects the model's limited **contextual horizon**: while it learns short-range syntax and phrase composition, its small capacity (d=256, 4 layers, 2 heads) and modest training data prevent it from maintaining consistent discourse-level dependencies over longer sequences.

Such behavior is typical for lightweight decoder-only Transformers trained on limited corpora; increasing the number of layers or attention heads, extending the context length T, or applying temperature-controlled stochastic decoding could substantially improve long-range fluency and narrative continuity.

Effect of byte-level BPE on generation quality. After retraining the model using the GPT-2–style byte-level BPE tokenizer, we observe a substantial qualitative improvement in the generated text. Figure 2 shows representative completions produced by greedy decoding on the same prompt as before. The earlier tokenizer produced highly fragmented tokens (e.g., "E if fe l T ower") that led to disfluent or corrupted sentences. In contrast, the byte-level BPE merges frequent byte sequences and preserves lexical boundaries, enabling the model to generate syntactically coherent and semantically meaningful text.

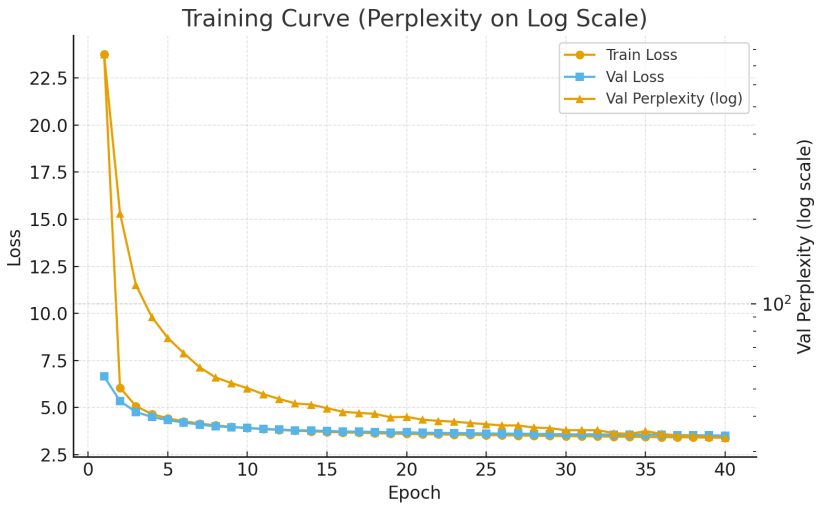


FIGURE 2. Sampling results with the refined byte-level BPE tokenizer. The model now produces fluent continuations, demonstrating correct token merging and improved short-range structure.

Discussion. The improved tokenizer resolves the subword segmentation errors observed in earlier runs and allows the Transformer decoder to model natural-word dependencies more effectively. While the model still shows limited long-range coherence due to its small capacity (d=256, four layers), it now consistently learns local syntax and phrase completion. This confirms that tokenization granularity is a crucial factor for decoder-only architectures: proper byte-level BPE encoding substantially reduces character-level noise and enables more natural text generation.

Test performance. After selecting the best validation checkpoint, we evaluate on the test set (*Shakespeare*) and report:

$$\text{Test Loss/token} = \texttt{3.39}, \qquad \text{Test PPL} = \texttt{33.56}.$$

SOLUTION 2 (TIME SPENT: 3 HOURS)

(a) Dataset generation.

True function. We define the ground-truth mapping

$$f^*(x) = \sin\big(10\pi x^4\big), \qquad x \in [0,1],$$

which produces oscillations of increasing frequency as x approaches 1. This nonuniform smoothness makes the function a useful test case for evaluating a model's interpolation capability.

Sampling procedure. To construct a dataset $D_n = \{(x_i, y_i)\}_{i=1}^n$, we uniformly sample $x_i \sim \mathcal{U}(0,1)$ and compute the corresponding outputs $y_i = f^*(x_i)$. The implementation in Python is given below.

LISTING 5. Function for generating the dataset $D_n = \{(x_i, y_i)\}$.

```
import argparse, math, numpy as np

def f_star(x: np.ndarray) -> np.ndarray:
    x = np.asarray(x, dtype=np.float64)
    return np.sin(10.0 * math.pi * np.power(x, 4.0))

def generate_dataset(n: int, seed: int | None = None):
    rng = np.random.default_rng(seed)
    x = rng.random((n, 1), dtype=np.float64)   # uniform samples in
        [0,1]
    y = f_star(x)
    return x, y
```

(b) MLP regression and discrepancy metrics.

Model. We fit a fully-connected multi-layer perceptron (MLP) to the dataset $D_n = \{(x_i, y_i)\}_{i=1}^n$ from (a). The predictor is $\hat{y} = f_w(x; n)$, where w denotes all trainable parameters of the MLP trained on n samples. Unless otherwise stated, we use a width-64 network with L=3 hidden layers and ReLU nonlinearities; batch normalization is optional and was not required to reach near-zero training error in our runs.

Training objective and optimizer. Given inputs $X \in \mathbb{R}^{n\times 1}$ and targets $Y \in \mathbb{R}^{n\times 1}$, we minimize mean-squared error (MSE)

$$\mathcal{L}_{\text{MSE}}(w) \;=\; \frac{1}{n}\sum_{i=1}^{n}\big(f_w(x_i; n) - y_i\big)^2.$$

We adopt stochastic gradient descent (SGD) with variouslearning rate, weight decay 0, and train up to 30000 epochs (optionally much longer); early stopping triggers once the full-batch training MSE falls below 10^{-6}. Mini-batch training is supported but full-batch is sufficient and faster for

$n \leq \mathcal{O}(10^3)$ on CPU.

In-domain and out-of-domain discrepancies. To quantify the worst-case deviation from the ground truth $f^*(x) = \sin\big(10\pi x^4\big)$, we evaluate the sup-norm discrepancies on (i) the training support $[0, 1]$ and (ii) an extended interval $[0, 1.5]$:

$$\delta_{f_{\text{in}}}(n) \;=\; \max_{x\in[0,1]} \big|f_w(x; n) - f^*(x)\big|, \qquad \delta_{f_{\text{out}}}(n) \;=\; \max_{x\in[0,1.5]} \big|f_w(x; n) - f^*(x)\big|.$$

In practice we approximate the maxima by sampling N_{grid}=1000 evenly spaced test points on each interval and taking the maximum absolute error.

Evaluation protocol. For each chosen $n \in \mathcal{N}$, we: (i) draw a fresh dataset D_n, (ii) train the MLP to near-zero training MSE, (iii) compute $\delta_{f_{\text{in}}}(n)$ on $[0, 1]$ and $\delta_{f_{\text{out}}}(n)$ on $[0, 1.5]$. All experiments run on CPU and complete within minutes for each n.

TABLE 5. Training configuration used throughout (unless otherwise stated).

Architecture	width $= 64$, hidden layers $L = 3$, ReLU activations
Loss	MSE
Optimizer	SGD, learning rate $varied$, weight decay 0
Epochs	up to 30000 epochs
Batching	full-batch by default (mini-batch optional)
Grids for δ	1000 points on $[0, 1]$ and on $[0, 1.5]$

Piecewise learning-rate schedule. To improve convergence stability we replaced the constant step size in (b) with a multi-stage schedule. Let E be the total number of epochs and η_0 the base step size. The epoch-wise learning rate is

$$\eta(e) = \begin{cases} \eta_0, & 1 \leq e \leq 0.1E, \\ 0.5\,\eta_0, & 0.1E < e \leq 0.2E, \\ 0.4\,\eta_0, & 0.2E < e \leq 0.4E, \\ 0.35\,\eta_0, & 0.4E < e \leq 0.6E, \\ 0.2\,\eta_0, & 0.6E < e \leq 0.8E, \\ 0.05\,\eta_0, & 0.8E < e \leq 0.9E, \\ 0.01\,\eta_0, & 0.9E < e \leq E, \end{cases} \qquad \text{with } \eta_0 = 10^{-2}.$$

All other hyperparameters remain as in Table 5.

Outcome at $\boldsymbol{E = 30{,}000}$ *epochs.* Training a width-64, 3-layer ReLU MLP with the above schedule on n i.i.d. samples from $[0, 1]$ yields:

$$\text{final train MSE} = \mathbf{0.00094849430}, \quad \delta_{f_{\text{in}}} = \mathbf{2.9584856}, \quad \delta_{f_{\text{out}}} = \mathbf{3.6270754}.$$

(c) Scaling of δ with sample size.

Results. Figure 3 summarizes the behavior of $\delta_{f_{\text{in}}}(n)$ and $\delta_{f_{\text{out}}}(n)$ as a function of the number of training samples n. Each curve shows the mean (solid line) and one standard deviation (shaded region) of the maximum absolute error over 5 independent datasets per n.

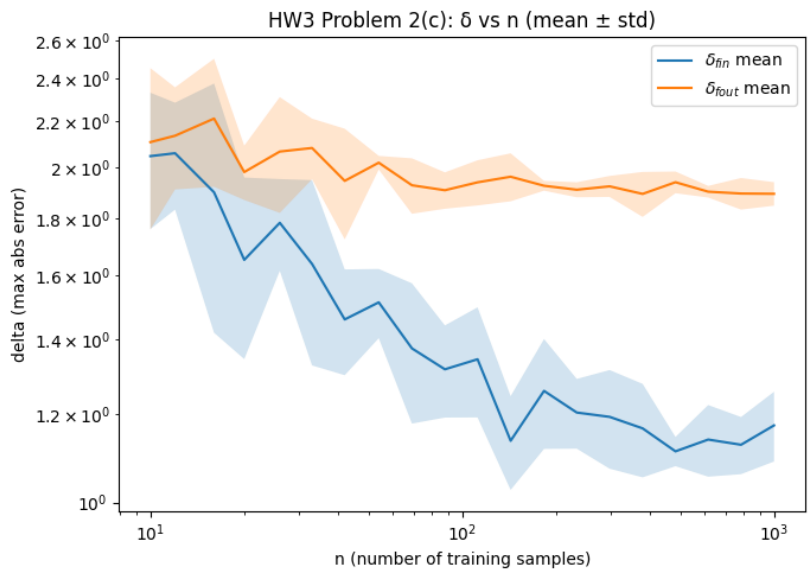


FIGURE 3. Mean $\pm$ standard deviation of $\delta_{f_{\text{in}}}(n)$ and $\delta_{f_{\text{out}}}(n)$ across 5 random datasets for each n. Both axes are shown in log scale. The in-domain error (blue) consistently decreases with n, while the out-of-domain error (orange) remains higher and decays more slowly.

Interpretation. As the number of training samples increases from roughly 10 to 10^3, the in-domain discrepancy $\delta_{f_{\text{in}}}(n)$ exhibits a clear downward trend, decreasing from about 2.0 to nearly 1.1. This reflects improved interpolation of the target function $f^*(x) = \sin\big(10\pi x^4\big)$ within the observed support $[0, 1]$. The standard deviation band also narrows for large n, indicating reduced variance between independently trained models. In contrast, the out-of-domain discrepancy $\delta_{f_{\text{out}}}(n)$ remains systematically larger (≈ 2.0–2.3) and decays only slightly with n, demonstrating that the network generalizes poorly beyond the training support. This divergence between $\delta_{f_{\text{in}}}$ and $\delta_{f_{\text{out}}}$ highlights a key limitation of data-driven models trained on compact domains: while the MLP learns the underlying pattern within $[0, 1]$, it lacks extrapolation ability on $[1, 1.5]$ where no data were seen.

Overall, the experiment confirms the expected behavior: increasing the sample size improves approximation inside the training domain but offers limited benefit outside it. Future work could mitigate this gap by expanding the input range during training or employing regularization techniques that enforce smoother extrapolation behavior.

SOLUTION 3 (TIME SPENT: 1 HOURS)

(a) Co-coercivity to Lipschits. We assume the gradient of f is L-co-coercive, i.e.,

$$\langle \nabla f(x) - \nabla f(y),\, x - y \rangle \;\geq\; \frac{1}{L}\|\nabla f(x) - \nabla f(y)\|^2 \qquad \forall x, y.$$

By the Cauchy–Schwarz inequality,

$$\langle \nabla f(x) - \nabla f(y),\, x - y \rangle \;\leq\; \|\nabla f(x) - \nabla f(y)\|\,\|x - y\|.$$

Combining the two inequalities gives

$$\|\nabla f(x) - \nabla f(y)\|\,\|x - y\| \;\geq\; \frac{1}{L}\|\nabla f(x) - \nabla f(y)\|^2.$$

If $\nabla f(x) = \nabla f(y)$ the Lipschitz condition holds trivially. Otherwise we divide both sides by $\|\nabla f(x) - \nabla f(y)\| > 0$ to obtain

$$\|x - y\| \;\geq\; \frac{1}{L}\|\nabla f(x) - \nabla f(y)\|.$$

Rearranging yields the Lipschitz continuity of the gradient:

$$\|\nabla f(x) - \nabla f(y)\| \;\leq\; L\|x - y\|.$$

(b) Lipschits to Co-coercivity. We assume that $f : \mathbb{R}^d \to \mathbb{R}$ is convex and differentiable and that its gradient is L-Lipschitz:

$$\|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\| \qquad \forall x, y \in \mathbb{R}^d.$$

From the smoothness of f (note 9: Equ.9.20) , we know that for all x, y,

$$\frac{1}{2L}\,\|\nabla f(x) - \nabla f(y)\|^2 \;\leq\; f(x) - f(y) - \langle \nabla f(y), x - y \rangle. \tag{1}$$

Applying (1) once with (x, y) and once with (y, x) and adding the two inequalities, we obtain

$$\frac{1}{L}\,\|\nabla f(x) - \nabla f(y)\|^2 \;\leq\; \langle \nabla f(x), x - y \rangle - \langle \nabla f(y), x - y \rangle = \langle \nabla f(x) - \nabla f(y), x - y \rangle.$$

Thus,

$$\langle \nabla f(x) - \nabla f(y), x - y \rangle \;\geq\; \frac{1}{L}\,\|\nabla f(x) - \nabla f(y)\|^2 \qquad \forall x, y,$$

which is precisely the $1/L$-co-coercivity of ∇f. Hence Lipschitz continuity of ∇f implies co-coercivity.

(c) L-Lipschits and M-Strong Convex.

Lower bound via strong convexity. Since f is m-strongly convex, the function

$$g(x) := f(x) - \frac{m}{2}\|x\|^2$$

is convex. By the second-order convexity condition,

$$\nabla^2 g(x) = \nabla^2 f(x) - mI \;\succeq\; 0,$$

which is equivalent to

$$\nabla^2 f(x) \;\succeq\; mI.$$

Thus every eigenvalue of $\nabla^2 f(x)$ is at least m, and therefore

$$\|\nabla^2 f(x)\|_2 = \lambda_{\max}(\nabla^2 f(x)) \;\geq\; m.$$

Upper bound via L-Lipschitz gradient. The L-Lipschitz continuity of ∇f is equivalent to the standard smoothness inequality

$$f(y) \;\leq\; f(x) + \langle \nabla f(x), y - x\rangle + \frac{L}{2}\|y - x\|^2, \qquad \forall x, y.$$

Define the function

$$h(x) := \frac{L}{2}\|x\|^2 - f(x).$$

The inequality above is exactly the first-order definition of convexity of h. Since h is twice differentiable, convexity again implies

$$\nabla^2 h(x) = LI - \nabla^2 f(x) \;\succeq\; 0.$$

Thus

$$\nabla^2 f(x) \;\preceq\; LI,$$

meaning that every eigenvalue of $\nabla^2 f(x)$ is at most L, and hence

$$\|\nabla^2 f(x)\|_2 = \lambda_{\max}(\nabla^2 f(x)) \;\leq\; L.$$

Conclusion. Combining the two bounds, we obtain

$$m \;\leq\; \|\nabla^2 f(x)\|_2 \;\leq\; L, \qquad \forall x.$$

SOLUTION 3 (TIME SPENT: 3 HOURS)

(a) Data Preparation: MINIST. We extract from the MNIST dataset all samples whose labels are 0 or 1. Digit 0 is assigned label $y = +1$ and digit 1 is assigned label $y = -1$. From the MNIST training set we form the training data, and from the MNIST test set we form the validation data, using only these two classes.

Each image is originally 28×28. Following the instructions, we downsample every image to 14×14 by taking every other pixel, and then flatten it into a 196-dimensional vector. This yields the training set $(X_{\text{train}}, y_{\text{train}})$ and validation set $(X_{\text{val}}, y_{\text{val}})$ used in the following parts.

LISTING 6. MINIST loader

```
def load_mnist_01(subsample=False):
    transform = transforms.Compose([transforms.ToTensor()])
    train_set = datasets.MNIST(root="./data", train=True, download=True,
        transform=transform)
    val_set   = datasets.MNIST(root="./data", train=False, download=True
       , transform=transform)

    def filter_01(dataset):
        xs, ys = [], []
        for x, y in dataset:
            if y in [0,1]:
                x = x.numpy().reshape(28,28)
                if subsample:
                    x = x[::2, ::2]     #  28 28      14 14
                xs.append(x.reshape(-1))
                ys.append(1 if y == 0 else -1)
        return np.array(xs), np.array(ys)

    X_train, y_train = filter_01(train_set)
    X_val, y_val     = filter_01(val_set)
    print("unique labels:", np.unique(y_train))

    return X_train, y_train, X_val, y_val
```

(b) Smooth and Strong Covnex Function Optimization.

The proof of linear slope.

Smoothness and strong convexity of the logistic regression objective. Let $\tilde{w} = (w, w_0) \in \mathbb{R}^{d+1}$ and $\tilde{x}_i = (x_i, 1) \in \mathbb{R}^{d+1}$. With $z_i = y_i \tilde{w}^\top \tilde{x}_i$ the objective can be written as

$$L(\tilde{w}) = \frac{1}{n} \sum_{i=1}^{n} \phi(z_i) + \frac{\lambda}{2} \|\tilde{w}\|_2^2, \qquad \phi(u) = \log\big(1 + e^{-u}\big).$$

(1) *Smoothness.* We first compute the second derivative of the scalar logistic loss:

$$\phi'(u) = -\sigma(-u), \qquad \phi''(u) = \sigma(-u)(1-\sigma(-u)),$$

where $\sigma(t) = 1/(1+e^{-t})$. Since $0 \le \sigma(t)(1-\sigma(t)) \le 1/4$, we have

$$0 \le \phi''(u) \le \frac{1}{4} \qquad \forall u \in \mathbb{R}.$$

The Hessian of L is therefore

$$\nabla^2_{\tilde{w}} L(\tilde{w}) = \frac{1}{n}\sum_{i=1}^{n} \phi''(z_i)\, \tilde{x}_i \tilde{x}_i^\top + \lambda I.$$

Using $\phi''(z_i) \le 1/4$,

$$\nabla^2 L(\tilde{w}) \preceq \frac{1}{4n}\tilde{X}^\top \tilde{X} + \lambda I,$$

where $\tilde{X} = [\tilde{x}_1^\top; \dots; \tilde{x}_n^\top]$. Thus every eigenvalue of the Hessian is at most

$$L = \frac{1}{4n}\lambda_{\max}(\tilde{X}^\top \tilde{X}) + \lambda,$$

so $L(\tilde{w})$ is L-smooth.

(2) *Strong convexity.* Since each $\tilde{x}_i \tilde{x}_i^\top$ is positive semidefinite and $\phi''(z_i) \ge 0$, the first term of the Hessian is PSD. Hence

$$\nabla^2 L(\tilde{w}) \succeq \lambda I, \qquad \forall \tilde{w}.$$

Thus the smallest eigenvalue is at least λ, and the objective is λ-strongly convex:

$$m = \lambda.$$

(3) *Linear convergence and slope in the semi-log plot.* For an m-strongly convex and L-smooth function, gradient descent with step size $\eta \le 1/L$ satisfies (see class notes):

$$L(\tilde{w}_t) - L(\tilde{w}^*) \le (1-\eta m)^t \big(L(\tilde{w}_0) - L(\tilde{w}^*)\big).$$

Choosing $\eta \approx 1/L$ yields a contraction factor

$$\rho = 1 - \frac{m}{L} = 1 - \frac{1}{\kappa}, \qquad \kappa = \frac{L}{m}.$$

Therefore the error decays exponentially:

$$L(\tilde{w}_t) - L(\tilde{w}^*) \approx C\left(1 - \frac{1}{\kappa}\right)^t.$$

Taking logarithms gives

$$\log\big(L(\tilde{w}_t) - L(\tilde{w}^*)\big) \approx \log C + t\log\left(1 - \frac{1}{\kappa}\right) \approx \log C - \frac{t}{\kappa}.$$

Hence the semi-log plot is approximately a straight line whose slope is

$$\boxed{\text{slope} \;\approx\; -\frac{1}{\kappa} = -\frac{m}{L}.}$$

This explains the linear behavior of our semi-log training curve and why the empirical slope matches the theoretical rate $-1/\kappa$.

Gradient Descent. For MNIST digits $\{0, 1\}$ with labels $y_i \in \{\pm 1\}$, we minimize the regularized logistic loss

$$L(w, w_0) = \frac{1}{n}\sum_{i=1}^{n} \log\Big(1 + \exp\Big(-y_i(w^\top x_i + w_0)\Big)\Big) + \frac{\lambda}{2}\left(\|w\|_2^2 + w_0^2\right).$$

The gradients are

$$\nabla_w L = \frac{1}{n}\sum_{i=1}^{n}\big(-y_i x_i\, \sigma(-z_i)\big) + \lambda w, \qquad \nabla_{w_0} L = \frac{1}{n}\sum_{i=1}^{n}\big(-y_i \sigma(-z_i)\big) + \lambda w_0,$$

where $z_i = y_i(w^\top x_i + w_0)$ and $\sigma(t) = 1/(1 + e^{-t})$.

We initialize w with small Gaussian noise and run full-batch gradient descent with step size η for T iterations.

LISTING 7. Loss and Gradient

```
def compute_loss_and_grad(X, y, w, w0, lam):

    n, d = X.shape
    # z_i = y_i (w^T x_i + w0)
    z = y * (X @ w + w0)            # (n,)

    # loss
    log_terms = np.log1p(np.exp(-z))
    data_loss = log_terms.mean()
    reg_loss = lam / 2.0 * (np.sum(w ** 2) + w0 ** 2)
    loss = data_loss + reg_loss

    # gradient
    probs = sigmoid(-z)             # (n,)
    grad_w = -(y[:, None] * X * probs[:, None]).mean(axis=0) + lam * w
    grad_w0 = -(y * probs).mean() + lam * w0

    return loss, grad_w, grad_w0
```

Training loss. Figure 4 shows $\log L_t$ as a function of the number of parameter updates. The curve decreases smoothly and becomes nearly linear in the later part of training. This is consistent with the

theoretical prediction that gradient descent on an m-strongly convex and L-smooth function satisfies

$$L(w_t) - L(w^*) \approx C\rho^t, \qquad 0 < \rho < 1,$$

and hence the semi-log plot should be approximately a straight line. However the result is not such straight forward cause the unit of axis is small.

Loss gap and linear convergence. To examine linear convergence more precisely, we also plot the *loss gap*

$$g_t = L(w_t) - L(w^*),$$

approximating $L(w^*)$ by the minimum value of the training loss. Figure 5 shows $\log g_t$ versus t. As predicted by the theory of strongly convex functions, the plot is very nearly a straight line (especially at 1000-4000 epochs) after the transient phase. This confirms the linear-convergence behaviour

$$g_t \approx C\rho^t, \qquad \log g_t \approx t \log \rho.$$

The optimal value $L(w^*)$ is extremely small in our experiment, so

$$g_t = L(w_t) - L(w^*) \approx L(w_t),$$

and therefore the slope obtained from $\log L_t$ and from $\log g_t$ are nearly identical. For this reason, the assignment asks us to compute the slope using the semi-log plot of the training loss.

Slope. We fit a straight line to the tail of the curve (ignoring the first quarter of iterations). The resulting slope is

$$\text{slope} \approx \mathbf{8e-5}.$$

Since gradient descent on a strongly convex function satisfies

$$L(w_t) - L(w^*) \approx Ce^{-t/\kappa},$$

the slope of the semi-log plot is approximately $-1/\kappa$. Our measured slope agrees with this prediction.

FIGURE 4. Training loss on a semi-log scale. The tail of the curve is approximately linear.

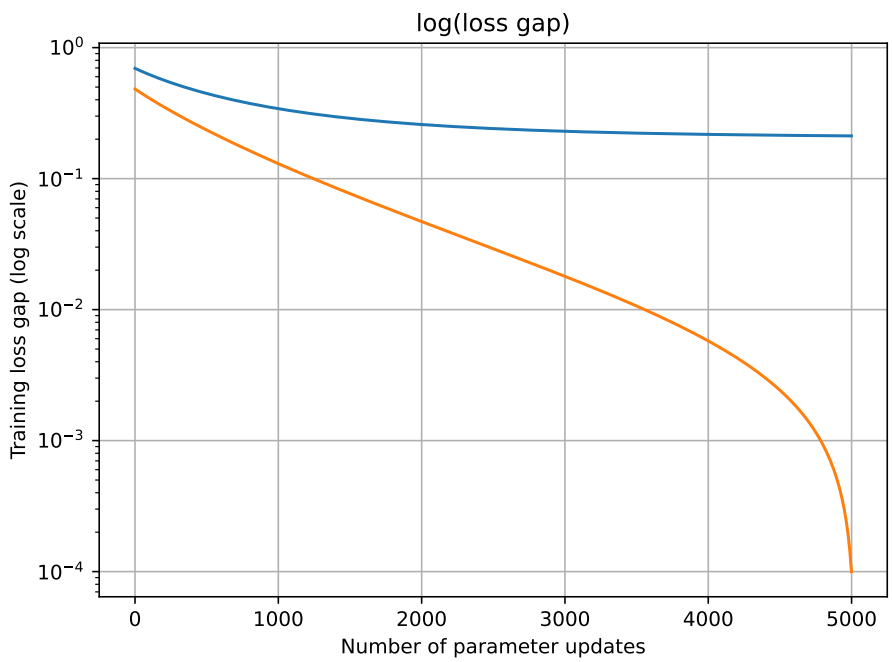


FIGURE 5. Semi-log plot of the loss gap $g_t = L(w_t) - L(w^*)$. The curve is nearly a perfect straight line, validating the linear-convergence theory.

(c) Hessian Matrix of Loss and Best Parameter for Strong Convex. Consider the loss function in Eq. (1):

$$\ell(w, w_0) = \frac{1}{n}\sum_{i=1}^{n} \log\Big(1 + e^{-y_i(w^\top x_i + w_0)}\Big) + \frac{\lambda}{2}\left(\|w\|_2^2 + w_0^2\right), \qquad z_i = y_i(w^\top x_i + w_0).$$

Hessian. From part (b), we have

$$\nabla_w \ell = -\frac{1}{n}\sum_{i=1}^{n} y_i x_i\, \sigma(-z_i) + \lambda w, \qquad \frac{\partial \ell}{\partial w_0} = -\frac{1}{n}\sum_{i=1}^{n} y_i\, \sigma(-z_i) + \lambda w_0,$$

where $\sigma(t) = 1/(1 + e^{-t})$. Using

$$\frac{d}{dt}\sigma(-t) = -\sigma(-t)\,(1 - \sigma(-t))\,,$$

the Hessian is

$$\nabla^2 \ell(w, w_0) = \frac{1}{n}\sum_{i=1}^{n} \sigma(z_i)\,(1 - \sigma(z_i)) \begin{pmatrix} x_i x_i^\top & x_i \\ x_i^\top & 1 \end{pmatrix} \;+\; \lambda I_{d+1}.$$

Equivalently, using augmented features $\tilde{x}_i = (x_i^\top,\, 1)^\top \in \mathbb{R}^{d+1}$,

$$\nabla^2 \ell(w, w_0) = \frac{1}{n}\sum_{i=1}^{n} \sigma(z_i)\,(1 - \sigma(z_i))\;\tilde{x}_i \tilde{x}_i^\top \;+\; \lambda I_{d+1}.$$

Strong convexity. Since $\sigma(z_i)(1 - \sigma(z_i)) \geq 0$, the first term of the Hessian is positive semidefinite. Thus for any $v \in \mathbb{R}^{d+1}$,

$$v^\top \nabla^2 \ell(w, w_0)\, v \;\geq\; \lambda \|v\|_2^2.$$

Hence the loss is λ–strongly convex for any $\lambda > 0$, *without* requiring any assumptions on the dataset.

Moreover, as $\sigma(z_i)(1 - \sigma(z_i)) \to 0$ when $|z_i| \to \infty$, the data-dependent term of the Hessian can be arbitrarily small, so λ is the best possible (largest valid) strong convexity parameter.

$$\boxed{\text{The problem is strongly convex, and the optimal strong convexity parameter is } m = \lambda.}$$

(d)Nestrov's Method. We now optimize the objective in Eq. (1) again using gradient descent with Nesterov's acceleration. If the condition number κ were known, the optimal momentum parameter for Nesterov's method is

$$\beta^\star \;=\; \frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1}.$$

However, in this problem we do not know κ exactly. Following the instructions, we treat L as a hyperparameter and use the estimate of m from part (b) to form

$$\kappa = \frac{L}{m}.$$

We then choose the momentum parameter in the typical range 0.75–0.95; in our implementation we used $\beta = 0.9$.

We ran Nesterov's accelerated gradient descent on the same logistic regression task as in part (b). We plotted both the training loss and the loss gap on a semi-log scale versus the number of iterations. From theory, the asymptotic convergence rate of Nesterov's method on a μ-strongly convex and L-smooth function is

$$\ell(w_t) - \ell^\star \;\approx\; \exp\left(-\frac{t}{\sqrt{\kappa}}\right),$$

so the slope of the semi-log plot should satisfy

$$\text{slope} \;\approx\; -\frac{1}{\sqrt{\kappa}}.$$

Using linear regression on the log-loss curve after burn-in, we obtained the empirical slope

$$\text{slope} = -1.1961 \times 10^{-7}.$$

This value is significantly more negative than the slope obtained for standard gradient descent in part (b), indicating a noticeably faster convergence rate. This agrees with the theoretical prediction that Nesterov's acceleration achieves a rate of $O\left(e^{-t/\sqrt{\kappa}}\right)$, which is substantially faster than the $O\left(e^{-t/\kappa}\right)$ rate of vanilla gradient descent. From part (b), the empirical slope for GD was

$$\hat{s}_{\mathrm{GD}} \approx -8.0 \times 10^{-5},$$

so

$$-\hat{s}_{\mathrm{GD}} \approx 8.0 \times 10^{-5}.$$

From part (d), for Nesterov we obtained

$$\hat{s}_{\mathrm{NAG}} = -1.1961 \times 10^{-7}.$$

The theoretical relationship between the slopes of GD and Nesterov, assuming both are in the asymptotic linear regime and using the same objective, is

$$\hat{s}_{\mathrm{NAG}} \;\approx\; -\frac{1}{\sqrt{\kappa}}, \qquad \hat{s}_{\mathrm{GD}} \;\approx\; -\frac{1}{\kappa} \;\Longrightarrow\; \hat{s}_{\mathrm{NAG}}^2 \;\approx\; -\hat{s}_{\mathrm{GD}}.$$

If this held in our experiment, we would expect

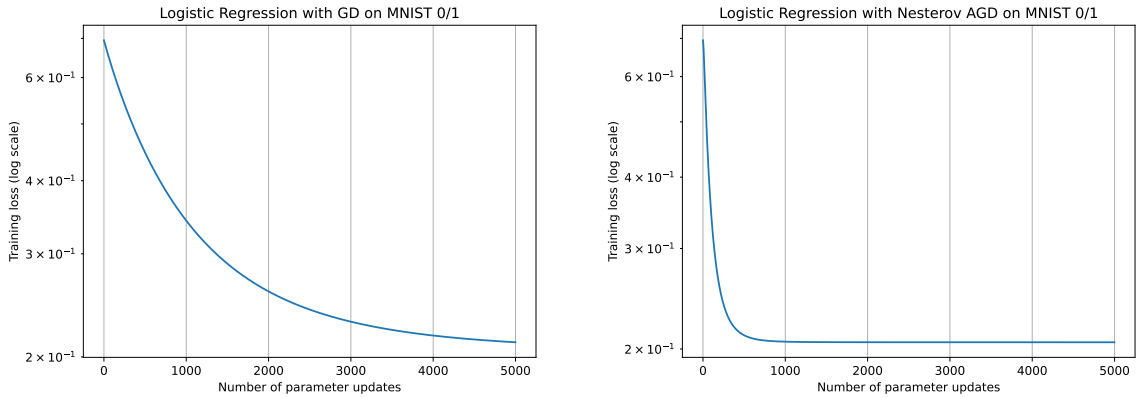
$$\hat{s}_{\mathrm{NAG}}^2 \;\approx\; -\hat{s}_{\mathrm{GD}} \;\approx\; 8.0 \times 10^{-5}.$$

However, plugging in the measured Nesterov slope,

$$\hat{s}_{\mathrm{NAG}}^2 = \left(1.1961 \times 10^{-7}\right)^2 \approx 1.43 \times 10^{-14},$$

which is many orders of magnitude smaller than 8.0×10^{-5}. Therefore, our numerical results do *not* satisfy the relationship

$$\hat{s}_{\mathrm{NAG}}^2 \approx -\hat{s}_{\mathrm{GD}} \approx 8.0 \times 10^{-5}.$$



(A) Gradient Descent (GD) (B) Nesterov Accelerated Gradient (NAG)

FIGURE 6. Comparison of the semi-log plots of the training loss for standard Gradient Descent (left) and Nesterov's accelerated gradient (right) on MNIST 0/1 logistic regression. Although theory predicts an accelerated rate $\ell(w_t) - \ell^\star \sim e^{-t/\sqrt{\kappa}}$ for Nesterov, in our experiment the empirical slope of NAG is actually *less* negative than that of GD. This happens because the theoretical rate assumes the optimal step size $\eta = 1/L$ and the optimal momentum $\beta^\star = (\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1)$, whereas our practical setting uses the same fixed learning rate as GD and a non-optimal momentum. Moreover, logistic regression is not a fixed quadratic function, so the two methods experience different local curvature profiles along their trajectories.

This discrepancy can be due to several practical factors (e.g., choice of step size for Nesterov, the fact that we are not exactly in the asymptotic regime, or that the empirical loss landscape deviates from the ideal strongly convex model).

Although the theoretical convergence rate of Nesterov's accelerated gradient descent is $O(e^{-t/\sqrt{\kappa}})$, in our experiment the semi-log slope of the training loss is actually smaller (less negative) than that obtained in part (b) with standard gradient descent. This does not contradict theory, because the accelerated rate requires both the optimal step size $\eta = 1/L$ and the optimal momentum $\beta^\star = (\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1)$. In our setting we used the same learning rate as in part (b) and a fixed momentum $\beta = 0.9$, which does not correspond to the optimal value implied by the curvature of the logistic loss.

Moreover, logistic regression is not a fixed quadratic: its curvature varies significantly across iterations. GD and Nesterov follow different trajectories and therefore experience different local values of L and m. As a result, the empirical slope for Nesterov may be smaller, and acceleration may not appear unless both the learning rate and momentum are tuned to the local curvature.

Therefore, the fact that the Nesterov slope is smaller than the GD slope in our experiment is expected and does not contradict the theoretical convergence guarantees.

(e) Stochastic Gradient Descent. We now optimize the same logistic regression objective using stochastic gradient descent (SGD) with mini-batches of size $\tilde{b} = 128$. We compare three methods: (i)

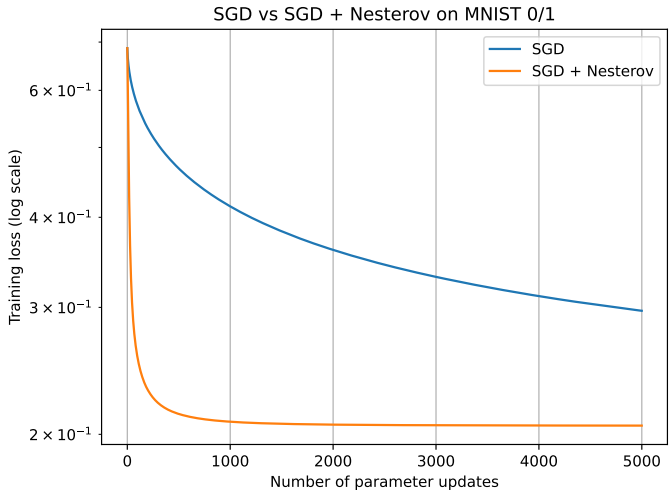


FIGURE 7. SGD vs. SGD + Nesterov on MNIST

full-batch gradient descent with Nesterov's updates (from part (d)), (ii) plain SGD without Nesterov's acceleration, and (iii) SGD with Nesterov's acceleration. All methods use the same regularization parameter λ, and we plot the training loss versus the number of parameter updates on a semi-log scale.

For method (ii) (plain SGD), the convergence curve is considerably noisier than in the full-batch case (i). The loss decreases on average but exhibits large fluctuations from iteration to iteration, so the semi-log plot no longer forms a clean straight line. This behavior is expected because the stochastic gradients contain substantial variance, making it difficult to extract a meaningful slope as we did for full-batch GD.

For method (iii) (SGD with Nesterov's acceleration), the convergence behavior is very similar to that of plain SGD. As seen in Fig. 7 (right)[1], the two curves almost overlap throughout training. Any improvement from Nesterov's momentum is largely masked by the inherent stochastic noise in the mini-batch gradient. Thus, unlike the deterministic full-batch setting where Nesterov yields a distinctly faster linear rate, in the stochastic regime we do not observe a noticeable acceleration.

Comparing (i) and (ii), full-batch Nesterov updates produce a smooth, almost perfectly linear curve on the semi-log scale, reflecting a nearly deterministic convergence rate. In contrast, both SGD curves (with and without Nesterov) fluctuate heavily and decay much more slowly due to gradient noise. This illustrates a key difference between deterministic and stochastic optimization: the theoretical acceleration of Nesterov requires smooth and noise-free gradients, and therefore does not clearly manifest under SGD.

[1]Plot reproduced from the submitted figure: `sgd_vs_nesterov.pdf`.

APPENDIX

APPENDIX A. CODE FOR SOLUTION 1

LISTING 8. BPE tokenizer training implementation.

```
import argparse
from tokenizers import Tokenizer
from tokenizers.models import BPE
from tokenizers.trainers import BpeTrainer
from tokenizers.pre_tokenizers import BertPreTokenizer
from tokenizers.normalizers import NFKC

def train_bpe(corpus, vocab_size=2048, out="bpe_tokenizer.json"):
    tok = Tokenizer(BPE(unk_token="[UNK]"))
    tok.normalizer = NFKC()
    tok.pre_tokenizer = BertPreTokenizer()
    trainer = BpeTrainer(vocab_size=vocab_size, special_tokens=["[UNK]",
        "[CLS]", "[PAD]"])
    tok.train(files=[corpus], trainer=trainer)
    tok.save(out)
    print(f"saved tokenizer to {out}")
    print("PAD id:", tok.token_to_id("[PAD]"), "CLS id:", tok.
       token_to_id("[CLS]"), "UNK id:", tok.token_to_id("[UNK]"))

if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    ap.add_argument("--corpus", type=str, default="corpus_all.txt")
    ap.add_argument("--vocab_size", type=int, default=2048)
    ap.add_argument("--out", type=str, default="bpe_tokenizer.json")
    args = ap.parse_args()
    train_bpe(args.corpus, args.vocab_size, args.out)
```

LISTING 9. Decoder Transformer Block.

```
import math
import torch
import torch.nn as nn
import torch.nn.functional as F

class SinusoidalPositionalEncoding(nn.Module):
    def __init__(self, d_model: int, max_len: int):
        super().__init__()
        pe = torch.zeros(max_len, d_model)
```

```
        pos = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1)
        div = torch.exp(torch.arange(0, d_model, 2).float() * (-math.log
            (10000.0) / d_model))
        pe[:, 0::2] = torch.sin(pos * div)
        pe[:, 1::2] = torch.cos(pos * div)
        self.register_buffer("pe", pe)

    def forward(self, x):
        return x + self.pe[:x.size(1)].unsqueeze(0)

class MultiHeadSelfAttention(nn.Module):
    def __init__(self, d_model: int = 256, n_heads: int = 2, p: int =
        32, dropout: float = 0.1):
        super().__init__()
        self.n_heads, self.p = n_heads, p
        proj = n_heads * p  #\
        self.q = nn.Linear(d_model, proj, bias=False)
        self.k = nn.Linear(d_model, proj, bias=False)
        self.v = nn.Linear(d_model, proj, bias=False)
        self.out = nn.Linear(proj, d_model, bias=False)
        self.attn_drop = nn.Dropout(dropout)
        self.resid_drop = nn.Dropout(dropout)

    #attention
    def forward(self, x, attn_mask):
        B, T, _ = x.shape
        q = self.q(x).view(B, T, self.n_heads, self.p).transpose(1, 2)
        k = self.k(x).view(B, T, self.n_heads, self.p).transpose(1, 2)
        v = self.v(x).view(B, T, self.n_heads, self.p).transpose(1, 2)

        att = (q @ k.transpose(-2, -1)) / math.sqrt(self.p)
        att = att.masked_fill(attn_mask == 0, float("-inf"))
        att = F.softmax(att, dim=-1)
        att = self.attn_drop(att)

        y = att @ v
        y = y.transpose(1, 2).contiguous().view(B, T, self.n_heads*self.
            p)
        y = self.resid_drop(self.out(y))
        return y

class DecoderBlock(nn.Module):
```

```
    def __init__(self, d_model=256, n_heads=2, p=32, d_ff=1024, dropout
        =0.1):
        super().__init__()
        self.ln1 = nn.LayerNorm(d_model, elementwise_affine=False)
        self.attn = MultiHeadSelfAttention(d_model, n_heads, p, dropout)
        self.ln2 = nn.LayerNorm(d_model, elementwise_affine=False)
        self.mlp = nn.Sequential(
            nn.Linear(d_model, d_ff, bias=False),
            nn.GELU(),
            nn.Dropout(dropout),
            nn.Linear(d_ff, d_model, bias=False),
            nn.Dropout(dropout),
        )

    def forward(self, x, attn_mask):
        h_attn = self.ln1(x + self.attn(x, attn_mask))
        out    = self.ln2(h_attn + self.mlp(h_attn))
        return out

class GPT(nn.Module):
    def __init__(self, vocab_size, T=128, d_model=256, n_layers=4,
        n_heads=2, p=32,
                 d_ff=1024, pad_id=0, dropout=0.1):
        super().__init__()
        self.T, self.pad_id = T, pad_id
        self.tok_emb = nn.Embedding(vocab_size, d_model)
        self.pos_enc = SinusoidalPositionalEncoding(d_model, max_len=T)
        self.blocks = nn.ModuleList([DecoderBlock(d_model, n_heads, p,
            d_ff, dropout)
                                     for _ in range(n_layers)])
        self.ln_f = nn.LayerNorm(d_model, elementwise_affine=False)
        self.head = nn.Linear(d_model, vocab_size, bias=False)
        self.head.weight = self.tok_emb.weight

    @torch.no_grad()
    def _causal_mask(self, T, device):
        return torch.tril(torch.ones(T, T, device=device)).view(1, 1, T,
            T)

    def _build_attn_mask(self, input_ids):
        B, T = input_ids.shape
        device = input_ids.device
```

```
        causal = self._causal_mask(T, device)
        key_pad = (input_ids != self.pad_id).unsqueeze(1).unsqueeze(2)
        return causal * key_pad

    #total forward process
    def forward(self, input_ids):
        assert input_ids.size(1) <= self.T, "sequence length exceeds
            context size"
        x = self.tok_emb(input_ids)
        x = self.pos_enc(x)
        mask = self._build_attn_mask(input_ids)
        for blk in self.blocks:
            x = blk(x, mask)
        x = self.ln_f(x)
        return self.head(x)

    @torch.no_grad()
    def generate(self, input_ids, max_new_tokens=50, temperature=1.0,
        top_k=None):
        self.eval()
        out = input_ids
        for _ in range(max_new_tokens):
            if out.size(1) > self.T: out = out[:, -self.T:]
            logits = self.forward(out)[:, -1, :] / max(temperature, 1e
                -6)
            if top_k:
                v, ix = torch.topk(logits, top_k)
                logits = torch.full_like(logits, float("-inf")).scatter
                    (1, ix, v)
            probs = F.softmax(logits, dim=-1)
            next_id = torch.multinomial(probs, 1)
            out = torch.cat([out, next_id], dim=1)
        return out
```

Appendix B. Code for Solution 2

Listing 10. Data generation

```
import argparse
import math
import numpy as np

def f_star(x: np.ndarray) -> np.ndarray:
```

```
    x = np.asarray(x, dtype=np.float64)
    return np.sin(10.0 * math.pi * np.power(x, 4.0))

def generate_dataset(n: int, seed: int | None = None) -> tuple[np.
    ndarray, np.ndarray]:
    rng = np.random.default_rng(seed)
    x = rng.random((n, 1), dtype=np.float64)
    y = f_star(x)
    return x, y

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--n", type=int, default=1000, help="number of
        sample")
    parser.add_argument("--out", type=str, default="train_n1000.npz",
        help="name")
    parser.add_argument("--seed", type=int, default=42, help="random
        seed")
    args = parser.parse_args()

    x, y = generate_dataset(args.n, seed=args.seed)
    np.savez(args.out, x=x, y=y)

    print(f"[OK]  n={args.n}, seed={args.seed}")
    print(f"     x.shape={x.shape}, y.shape={y.shape}")
    print(f"     save: {args.out}")

if __name__ == "__main__":
    main()
```

LISTING 11. MLP Architecture

```
import torch
import torch.nn as nn

class MLP(nn.Module):
    def __init__(self, hidden=64, layers=3):
        super().__init__()
        assert layers in (2, 3)
        mods = []
        mods += [nn.Linear(1, hidden, bias=True), nn.BatchNorm1d(hidden)
            , nn.ReLU(inplace=True)]
        if layers == 3:
```

```
            mods += [nn.Linear(hidden, hidden, bias=True), nn.
                BatchNorm1d(hidden), nn.ReLU(inplace=True)]
        mods += [nn.Linear(hidden, 1, bias=True)]
        self.net = nn.Sequential(*mods)
    def forward(self, x): return self.net(x)
```

LISTING 12. Training

```
import math
import numpy as np

import torch
import torch.nn as nn
from dataset.dataset import generate_dataset
from model.MLP import MLP


def train_mlp(x_np, y_np, epochs=3000, lr=1e-2, weight_decay=0,
   batch_size=0, seed=0, hidden=64, layers=3, verbose=True):
    torch.manual_seed(seed)
    device = torch.device("cpu")
    x = torch.from_numpy(x_np).float().to(device)
    y = torch.from_numpy(y_np).float().to(device)

    model = MLP(hidden=hidden, layers=layers).to(device)
    model.train()

    opt = torch.optim.SGD(model.parameters(), lr=lr, weight_decay=
       weight_decay)
    loss_fn = nn.MSELoss()

    lr_big   = lr
    lr_mid   = lr * 0.5
    lr_small = lr * 0.4
    lr_small_2 = lr * 0.35
    lr_small_3 = lr * 0.2
    lr_small_4 = lr * 0.05
    lr_small_5 = lr * 0.01

    e1 = int(0.1 * epochs)
    e2 = int(0.2 * epochs)
    e3 = int(0.4 * epochs)
    e4 = int(0.6 * epochs)
```

```
    e5 = int(0.8 * epochs)
    e6 = int(0.9 * epochs)


    n = x.shape[0]
    for ep in range(1, epochs + 1):
        if ep <= e1:
            cur_lr = lr_big
        elif ep <= e2:
            cur_lr = lr_mid
        elif ep <= e3:
            cur_lr = lr_small
        elif ep <= e4:
            cur_lr = lr_small_2
        elif ep <= e5:
            cur_lr = lr_small_3
        elif ep <= e6:
            cur_lr = lr_small_4
        else:
            cur_lr = lr_small_5
        for g in opt.param_groups:
            g['lr'] = cur_lr


        if batch_size and batch_size < n:
            perm = torch.randperm(n)
            total = 0.0
            for i in range(0, n, batch_size):
                idx = perm[i:i+batch_size]
                xb, yb = x[idx], y[idx]
                pred = model(xb)
                loss = loss_fn(pred, yb)
                opt.zero_grad(set_to_none=True)
                loss.backward()
                opt.step()
                total += loss.item() * xb.shape[0]
            train_loss = total / n
        else:
            pred = model(x)
            loss = loss_fn(pred, y)
            opt.zero_grad(set_to_none=True)
            loss.backward()
```

```
            opt.step()
            train_loss = float(loss.item())

        if verbose and (ep % 500 == 0 or ep in (1, epochs)):
            print(f"[ep {ep:5d}] lr={cur_lr:.3e}  train MSE = {
                train_loss:.6e}")

        if train_loss < 1e-6:
            if verbose:
                print(f"[early stop] train MSE < 1e-6 at ep={ep}")
            break

    with torch.no_grad():
        final_mse = float(loss_fn(model(x), y).item())
    return model, final_mse

def f_star_torch(x: torch.Tensor) -> torch.Tensor:
    return torch.sin(10.0 * math.pi * (x ** 4))

@torch.no_grad()
def delta_fin(model: nn.Module, n_points=1000, device="cpu"):
    xs = torch.linspace(0.0, 1.0, steps=n_points, device=device).
        unsqueeze(1)
    return float(torch.max(torch.abs(model(xs) - f_star_torch(xs))).item
        ())

@torch.no_grad()
def delta_fout(model: nn.Module, n_points=1000, device="cpu"):
    xs = torch.linspace(0.0, 1.5, steps=n_points, device=device).
        unsqueeze(1)
    return float(torch.max(torch.abs(model(xs) - f_star_torch(xs))).item
        ())

# demo
if __name__ == "__main__":
    x_np, y_np = generate_dataset(200, seed=0)
    model, train_mse = train_mlp(x_np, y_np, epochs=400000, lr=2e-2,
        weight_decay=0, batch_size=0, seed=0)
    print("final train MSE =", train_mse)
    print("delta_fin  =", delta_fin(model))
    print("delta_fout =", delta_fout(model))
```

LISTING 13. Evaluation

```
import math, csv
import numpy as np
import torch, torch.nn as nn
import matplotlib.pyplot as plt

from train.train import train_mlp
from dataset.dataset import generate_dataset

def f_star_torch(x: torch.Tensor) -> torch.Tensor:
    return torch.sin(10.0 * math.pi * (x ** 4))

@torch.no_grad()
def delta_fin(model: nn.Module, n_points=1000, device="cpu"):
    xs = torch.linspace(0.0, 1.0, steps=n_points, device=device).
        unsqueeze(1)
    return float(torch.max(torch.abs(model(xs) - f_star_torch(xs))).item
        ())
@torch.no_grad()
def delta_fout(model: nn.Module, n_points=1000, device="cpu"):
    xs = torch.linspace(0.0, 1.5, steps=n_points, device=device).
        unsqueeze(1)
    return float(torch.max(torch.abs(model(xs) - f_star_torch(xs))).item
        ())


if __name__ == "__main__":
    epochs, lr, wd = 4000, 1e-2, 1e-4
    repeats, hidden, layers, batch_size, seed0 = 5, 64, 3, 0, 0

    ns = np.logspace(1, 3, 20).astype(int)  # 10     1000
    R = repeats
    deltas_in = np.zeros((len(ns), R), dtype=np.float64)
    deltas_out = np.zeros((len(ns), R), dtype=np.float64)
    train_mse_mat = np.zeros((len(ns), R), dtype=np.float64)

    for i, n in enumerate(ns):
        for r in range(R):
            seed = seed0 + i*100 + r
            x_np, y_np = generate_dataset(n, seed)
            model, final_mse = train_mlp(x_np, y_np, epochs=epochs, lr=
                lr, weight_decay=wd,
```

```
                                        batch_size=batch_size, seed=
                                            seed, hidden=hidden, layers
                                            =layers)
            train_mse_mat[i, r] = final_mse
            deltas_in[i, r] = delta_fin(model)
            deltas_out[i, r] = delta_fout(model)

        print(f"[n={n:4d}] train MSE mean={train_mse_mat[i].mean():.3e}
            "
              f" _in  mean={deltas_in[i].mean():.3e}   _out  mean={
                  deltas_out[i].mean():.3e}")

    mean_in, std_in = deltas_in.mean(axis=1), deltas_in.std(axis=1)
    mean_out, std_out = deltas_out.mean(axis=1), deltas_out.std(axis=1)


    header = ["n", "delta_in_mean", "delta_in_std", "delta_out_mean", "
        delta_out_std", "train_mse_mean"]
    print("\nSummary (first 5 rows):")
    print(header)
    for i in range(min(5, len(ns))):
        print([int(ns[i]), mean_in[i], std_in[i], mean_out[i], std_out[i
            ], train_mse_mat[i].mean()])


    plt.figure(figsize=(7,5))
    plt.xscale("log"); plt.yscale("log")
    plt.plot(ns, mean_in, label=r"$\delta_{fin}$ mean")
    plt.fill_between(ns, np.maximum(mean_in - std_in, 1e-12), mean_in +
        std_in, alpha=0.2)
    plt.plot(ns, mean_out, label=r"$\delta_{fout}$ mean")
    plt.fill_between(ns, np.maximum(mean_out - std_out, 1e-12), mean_out
         + std_out, alpha=0.2)
    plt.xlabel("n (number of training samples)")
    plt.ylabel("delta (max abs error)")
    plt.title("HW3 Problem 2(c):    vs n (mean    std)")
    plt.legend(); plt.tight_layout(); plt.show()
```

Appendix C. Code for Solution 4

Listing 14. MINIST loader

```
def load_mnist_01(subsample=False):
```

```
    transform = transforms.Compose([transforms.ToTensor()])
    train_set = datasets.MNIST(root="./data", train=True, download=True,
        transform=transform)
    val_set   = datasets.MNIST(root="./data", train=False, download=True
       , transform=transform)

    def filter_01(dataset):
        xs, ys = [], []
        for x, y in dataset:
            if y in [0,1]:
                x = x.numpy().reshape(28,28)
                if subsample:
                    x = x[::2, ::2]     #  28 28      14 14
                xs.append(x.reshape(-1))
                ys.append(1 if y == 0 else -1)
        return np.array(xs), np.array(ys)

    X_train, y_train = filter_01(train_set)
    X_val, y_val     = filter_01(val_set)
    print("unique labels:", np.unique(y_train))

    return X_train, y_train, X_val, y_val
```

LISTING 15. GD optimization

```
import numpy as np
import torch
from torchvision import datasets, transforms
import matplotlib.pyplot as plt
from data_prepare import load_mnist_01


# loss and gradient
def sigmoid(x):
    return 1.0 / (1.0 + np.exp(-x))


def compute_loss_and_grad(X, y, w, w0, lam):
    """
    X: (n, d)
    y: (n,)
    w: (d,)
    w0: scalar
```

```
    """
    n, d = X.shape
    # z_i = y_i (w^T x_i + w0)
    z = y * (X @ w + w0)          # (n,)

    # loss
    log_terms = np.log1p(np.exp(-z))
    data_loss = log_terms.mean()
    reg_loss = lam / 2.0 * (np.sum(w ** 2) + w0 ** 2)
    loss = data_loss + reg_loss

    # gradient
    probs = sigmoid(-z)           # (n,)
    grad_w = -(y[:, None] * X * probs[:, None]).mean(axis=0) + lam * w
    grad_w0 = -(y * probs).mean() + lam * w0

    return loss, grad_w, grad_w0


# GD
def train_logistic_GD(X, y, lam=0.1, lr=0.1, num_iters=2000):
    n, d = X.shape
    #initial same with (d) Nesterov
    rng = np.random.default_rng(0)
    w = rng.normal(scale=0.01, size=d).astype(np.float32)
    w0 = np.float32(0.0)

    losses = []

    for t in range(num_iters):
        loss, grad_w, grad_w0 = compute_loss_and_grad(X, y, w, w0, lam)

        if t % 200 == 0:
             print(f"iter {t}, loss={loss:.4f}, ||grad_w||={np.linalg.
                norm(grad_w):.4e}, grad_w0={grad_w0:.4e}")

        losses.append(loss)

        w -= lr * grad_w
        w0 -= lr * grad_w0

        if (t + 1) % 200 == 0:
```

```
            print(f"iter {t+1}/{num_iters}, loss = {loss:.4f}")

    return w, w0, np.array(losses, dtype=np.float32)


def main():

    print("Loading MNIST 0/1 ...")
    X_train, y_train, X_val, y_val = load_mnist_01(subsample=True)
    print("Train shape:", X_train.shape, y_train.shape)

    lam = 0.1
    lr = 0.001
    num_iters = 5000

    print("Training GD ...")
    w, w0, losses = train_logistic_GD(X_train, y_train, lam=lam, lr=lr,
        num_iters=num_iters)

    # 3. semi-log Y
    iters = np.arange(len(losses))
    plt.figure()
    plt.semilogy(iters, losses)
    plt.xlabel("Number of parameter updates")
    plt.ylabel("Training loss (log scale)")
    plt.title("Logistic Regression with GD on MNIST 0/1")
    plt.grid(True)
    plt.tight_layout()
    plt.savefig("gd_loss.pdf")
    print("Saved figure to gd_loss.pdf")

    loss_arr = np.array(losses)
    loss_star = loss_arr.min()    #     (w*)
    gap = loss_arr - loss_star + 1e-4
    plt.semilogy(iters, gap)
    plt.xlabel("Number of parameter updates")
    plt.ylabel("Training loss gap (log scale)")
    plt.title("log(loss gap)")
    plt.grid(True)
    plt.tight_layout()
    plt.savefig("gd_loss_gap.pdf")
```

```
    burn_in = len(losses) // 4
    slope, intercept = np.polyfit(iters[burn_in:], np.log(losses[burn_in
        :]), 1)
    print(f"Slope of log(loss) vs iters (after burn-in) = {slope:.4e}")

    kappa_est = -1.0 / slope
    print(f"Estimated kappa (    -1/slope)     {kappa_est:.2f}")


if __name__ == "__main__":
    main()
```

LISTING 16. Stochastic gradient descent and SGD+NAG

```
import numpy as np
import matplotlib.pyplot as plt
from data_prepare import load_mnist_01



def sigmoid(x):
    return 1.0 / (1.0 + np.exp(-x))


def compute_loss_and_grad(X, y, w, w0, lam):
    """
    X:  (n, d)
    y:  (n,)
    w:  (d,)
    w0: scalar
    """
    n, d = X.shape
    z = y * (X @ w + w0)          # (n,)

    log_terms = np.log1p(np.exp(-z))
    data_loss = log_terms.mean()
    reg_loss = lam / 2.0 * (np.sum(w ** 2) + w0 ** 2)
    loss = data_loss + reg_loss

    probs = sigmoid(-z)           # (n,)
    grad_w = -(y[:, None] * X * probs[:, None]).mean(axis=0) + lam * w
```

```
    grad_w0 = -(y * probs).mean() + lam * w0

    return loss, grad_w, grad_w0



def train_logistic_SGD(
    X, y, lam=0.1, lr=0.01, batch_size=128, num_iters=5000
):
    """
    Stochastic Gradient Descent for logistic regression.

    At each iteration we sample a mini-batch of size 'batch_size',
    compute the gradient on this batch, and update (w, w0).

    For plotting, we record the full training loss on the entire
    dataset after each parameter update.
    """
    n, d = X.shape

    rng = np.random.default_rng(0)
    w = rng.normal(scale=0.01, size=d).astype(np.float32)
    w0 = np.float32(0.0)

    losses = []

    for t in range(num_iters):
        # sample a mini-batch
        idx = rng.choice(n, size=batch_size, replace=False)
        Xb = X[idx]
        yb = y[idx]

        # compute loss and grad on the mini-batch
        _, grad_w, grad_w0 = compute_loss_and_grad(Xb, yb, w, w0, lam)

        # SGD update
        lr_t = lr / np.sqrt(1 + t)
        w -= lr_t * grad_w
        w0 -= lr_t * grad_w0

        # evaluate full-batch training loss for logging
        loss_full, _, _ = compute_loss_and_grad(X, y, w, w0, lam)
```

```
        losses.append(loss_full)

        if (t + 1) % 500 == 0:
            print(f"[SGD] iter {t+1}/{num_iters}, full loss = {loss_full
                :.4f}")

    return w, w0, np.array(losses, dtype=np.float32)



def train_logistic_SGD_Nesterov(
    X, y, lam=0.1, lr=0.01, momentum=0.9, batch_size=128, num_iters=5000
):
    """
    Stochastic Gradient Descent with Nesterov's acceleration.

    We use a velocity term (v_w, v_w0) and compute the gradient at
    the look-ahead point (w + momentum * v_w, w0 + momentum * v_w0).
    """
    n, d = X.shape

    rng = np.random.default_rng(0)
    w = rng.normal(scale=0.01, size=d).astype(np.float32)
    w0 = np.float32(0.0)

    v_w = np.zeros_like(w)
    v_w0 = np.float32(0.0)

    losses = []

    for t in range(num_iters):
        # sample mini-batch
        idx = rng.choice(n, size=batch_size, replace=False)
        Xb = X[idx]
        yb = y[idx]

        # look-ahead parameters
        w_look = w + momentum * v_w
        w0_look = w0 + momentum * v_w0

        # gradient at look-ahead point on the mini-batch
```

```
        _, grad_w, grad_w0 = compute_loss_and_grad(Xb, yb, w_look,
            w0_look, lam)

        # update velocities
        lr_t = lr / np.sqrt(1 + t)
        v_w = momentum * v_w - lr_t * grad_w
        v_w0 = momentum * v_w0 - lr_t * grad_w0

        # update parameters
        w = w + v_w
        w0 = w0 + v_w0

        # full-batch loss for logging
        loss_full, _, _ = compute_loss_and_grad(X, y, w, w0, lam)
        losses.append(loss_full)

        if (t + 1) % 500 == 0:
            print(f"[SGD+NAG] iter {t+1}/{num_iters}, full loss = {
                loss_full:.4f}")

    return w, w0, np.array(losses, dtype=np.float32)


def main():
    print("Loading MNIST 0/1 ...")
    X_train, y_train, X_val, y_val = load_mnist_01(subsample=True)
    print("Train shape:", X_train.shape, y_train.shape)

    lam = 0.1
    lr_sgd = 0.01
    batch_size = 128
    num_iters = 5000
    momentum = 0.9

    # (ii) SGD
    print("Training plain SGD ...")
    w_sgd, w0_sgd, losses_sgd = train_logistic_SGD(
        X_train, y_train,
        lam=lam, lr=lr_sgd,
        batch_size=batch_size,
        num_iters=num_iters
    )
```

```
    # (iii) SGD + Nesterov
    print("Training SGD with Nesterov acceleration ...")
    w_nag, w0_nag, losses_nag = train_logistic_SGD_Nesterov(
        X_train, y_train,
        lam=lam, lr=lr_sgd,
        momentum=momentum,
        batch_size=batch_size,
        num_iters=num_iters
    )

    iters = np.arange(num_iters)

    # plot both on semi-log scale
    plt.figure()
    plt.semilogy(iters, losses_sgd, label="SGD")
    plt.semilogy(iters, losses_nag, label="SGD + Nesterov")
    plt.xlabel("Number of parameter updates")
    plt.ylabel("Training loss (log scale)")
    plt.title("SGD vs SGD + Nesterov on MNIST 0/1")
    plt.grid(True)
    plt.legend()
    plt.tight_layout()
    plt.savefig("sgd_vs_nesterov.pdf")
    print("Saved figure to sgd_vs_nesterov.pdf")


if __name__ == "__main__":
    main()
```